UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA
DELL'INFORMAZIONE

Laurea Triennale in Ingegneria Informatica

Sicurezza delle applicazioni web: studio e dimostrazione pratica di vulnerabilità nel caricamento dei file

Relatore:
Prof. Pericle Perazzo

Candidato:
Canessa Matteo

Anno Accademico 2025/2026

Indice

Introduzione **2**

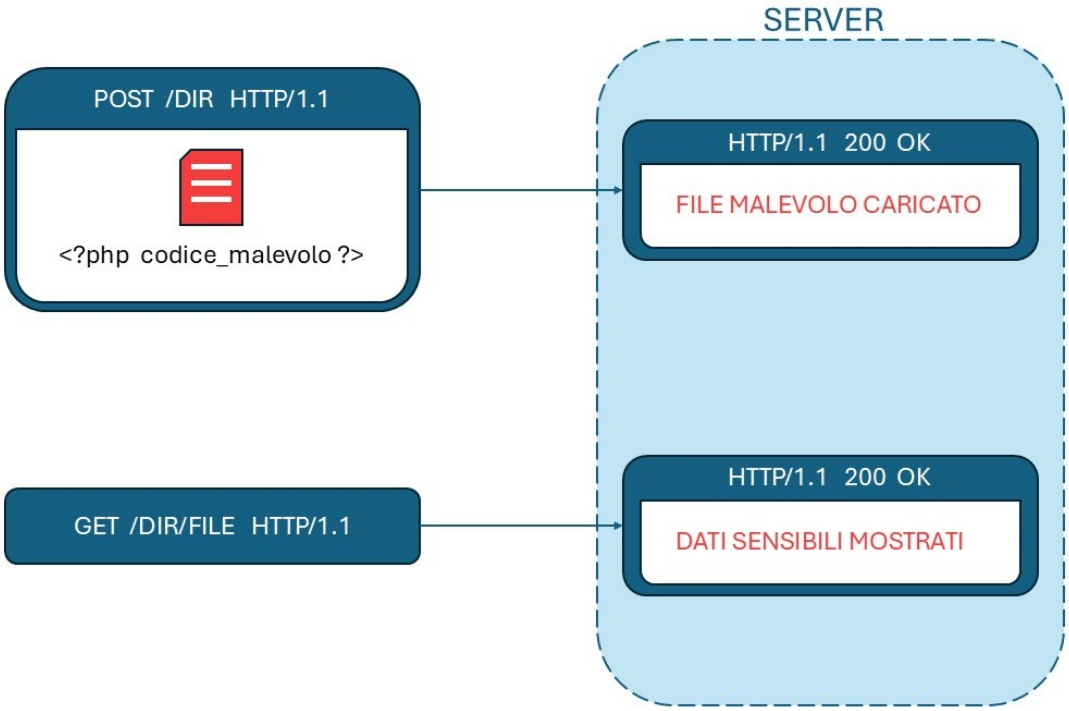
Scenario della challenge **3**

Caso base **4**
Sfruttamento delle vulnerabilità 5

Caso medio **8**
Creazione di file poliglotti 9
Sfruttamento delle vulnerabilità 9

Introduzione

Durante lo sviluppo di applicazioni web è fondamentale considerare gli aspetti legati alla sicurezza. Tuttavia, nei progetti di piccole dimensioni o in contesti caratterizzati da limitata esperienza nello sviluppo, questi aspetti vengono spesso trascurati. Una delle aree più critiche in questo ambito riguarda la gestione del caricamento dei file da parte degli utenti.



In questo documento verranno analizzate due vulnerabilità relative ai meccanismi di file upload, caratterizzate da livelli di complessità differenti. Successivamente verrà dimostrato come tali vulnerabilità possano essere sfruttate in uno scenario realistico e quali contromisure possano essere adottate per prevenirle.

Per studiare questo scenario è stata sviluppata da zero un'applicazione web dotata di una funzionalità di caricamento file. L'applicazione è stata progettata includendo deliberatamente alcune vulnerabilità, con l'obiettivo di analizzarne il funzionamento e dimostrarne lo sfruttamento pratico.

Per garantire la portabilità del progetto è stato utilizzato un ambiente di sviluppo basato su Docker. Sono state utilizzate immagini ufficiali di PHP e Apache per creare un ambiente di esecuzione isolato e replicabile. Questo approccio consente di eseguire l'applicazione su qualsiasi macchina senza la necessità di configurare manualmente le dipendenze o l'ambiente di esecuzione.

Inoltre sono stati utilizzati alcuni strumenti per lo svolgimento pratico degli attacchi:

- **Burp Suite**: un tool di sicurezza web che consente di intercettare e modificare le richieste HTTP, facilitando l'identificazione e lo sfruttamento delle vulnerabilità.
- **ExifTool**: un software che permette di visualizzare e modificare i metadati dei file, utile per inserire payload malevoli all'interno dei file caricati.

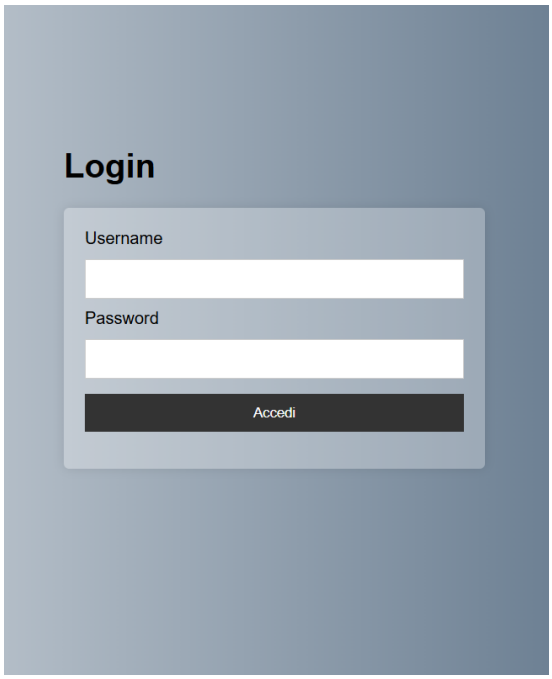


Figure 1: Pagina di login

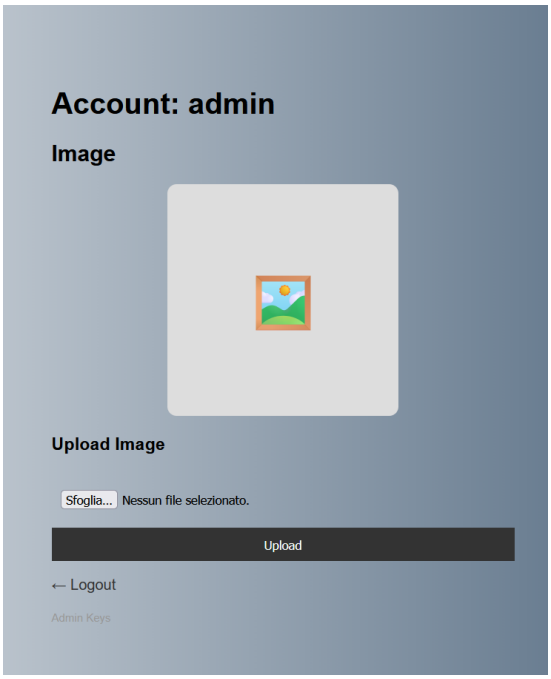


Figure 2: Pagina di upload

Scenario della challenge

Per dimostrare in modo pratico lo sfruttamento delle vulnerabilità analizzate è stato realizzato uno scenario ispirato alle competizioni di sicurezza informatica note come Capture The Flag (CTF).

L'applicazione utilizzata nello scenario prende il nome di "ImageShare Pro" e simula una piattaforma di caricamento e condivisione di immagini.

Durante il deploy di una nuova versione dell'applicazione è stata introdotta una vulnerabilità nella gestione dei file caricati dagli utenti.

Secondo alcune informazioni interne, l'amministratore del sistema ha memorizzato la Master Key utilizzata per accedere alle immagini private degli utenti all'interno del file:

`/var/www/config/database.yml`

L'obiettivo dell'attaccante è quindi sfruttare le vulnerabilità presenti nell'applicazione per ottenere l'accesso a tale chiave.

Per semplificare lo scenario, si assume che l'attaccante abbia già un account valido sulla piattaforma e sia in grado di effettuare l'accesso (username: `admin`, password: `admin`); è stato inserito un forbidden link alla pagina di upload per mostrare il path del file che contiene la Master Key.

Caso base

Una volta effettuato l'accesso, l'utente può raggiungere una pagina personale dalla quale è possibile caricare file all'interno della directory `files/images/`.

A prima vista, questa funzionalità rappresenta un'implementazione comune presente in molte applicazioni web moderne. Tuttavia, l'applicazione presenta due vulnerabilità fondamentali che, se combinate, permettono a un attaccante di compromettere completamente il sistema.

La prima vulnerabilità è nota come *Unrestricted File Upload*. In questo caso il codice dell'applicazione non effettua alcun controllo sul tipo di file caricato. Sebbene l'interfaccia utente suggerisca implicitamente che siano accettate soltanto immagini, il server accetta in realtà qualsiasi tipologia di file, inclusi file PHP contenenti codice eseguibile.

```php
<?php
// Esempio di codice PHP che causa la vulnerabilità di Unrestricted File Upload
if (isset($_FILES['file'])) {
    $file = $_FILES['file'];
    $filename = basename($file['name']);
}
?>
```

Il controllo effettuato lato client può essere facilmente aggirato e, in assenza di una validazione rigorosa lato server, un attaccante può caricare una web shell PHP che consente l'esecuzione di comandi arbitrari sul sistema.

Un ulteriore problema è rappresentato dalla configurazione del file `.htaccess` presente nella directory di destinazione. Tale file dovrebbe impedire l'esecuzione di script PHP all'interno della cartella dove viene salvato il file; tuttavia, un errore di sintassi nella configurazione ne compromette l'efficacia, rendendo la directory vulnerabile all'esecuzione di codice lato server.

```
# Contenuto del file .htaccess
# Configurazione che permette l'esecuzione di file PHP
<FilesMatch "\.php$">
    SetHandler application/x-httpd-php
</FilesMatch>
```

La seconda vulnerabilità riguarda invece la gestione della sanitizzazione del nome del file caricato. Nel file `upload.php` il codice tenta di prevenire attacchi di *Path Traversal* rimuovendo la sequenza `../` dal nome del file caricato. Tale sequenza consentirebbe infatti di risalire la gerarchia delle directory e scrivere file al di fuori della cartella prevista.

Tuttavia, la procedura di sanitizzazione viene eseguita prima della decodifica URL del nome del file. Questo comportamento introduce una vulnerabilità: un attaccante può utilizzare la codifica URL per mascherare la sequenza `../`, trasformandola ad esempio in `..%2f`.

Il filtro non riconosce questa sequenza come pericolosa e la lascia passare. Successivamente, la funzione `rawurldecode()` [1] decodifica il valore, trasformandolo nuovamente in

[1] La funzione rawurldecode() decodifica stringhe URL-encoded secondo lo standard RFC 3986, a differenza di urldecode() che usa il formato application/x-www-form-urlencoded

`../` e rendendo possibile l'attacco di Path Traversal.

L'attacco di Path Traversal viene utilizzato per posizionare il codice malevolo in una directory accessibile dal server web, nella quale i file PHP vengono eseguiti.

```php
<?php
    // Esempio di codice PHP che causa la vulnerabilità di Path Traversal
    $filename = str_replace('../', '', $filename);

    // La decodifica URL avviene dopo la sanitizzazione,
    // permettendo di aggirare il filtro
    $filename = rawurldecode($filename);
?>
```

Sfruttamento delle vulnerabilità

Lo sfruttamento delle vulnerabilità nel caso base può avvenire attraverso un approccio diretto basato sul caricamento di una web shell PHP e sull'utilizzo di path traversal per posizionarla in una directory accessibile. In questa configurazione sono stati resi possibili entrambi gli approcci per semplificare la dimostrazione.

Approccio 1: Caricamento della web shell

Il primo passo consiste nel caricare un file PHP contenente codice malevolo attraverso l'interfaccia di upload dell'applicazione. Poiché non sono presenti controlli sulla tipologia di file, è possibile caricare direttamente un file con estensione `.php` senza alcuna limitazione.

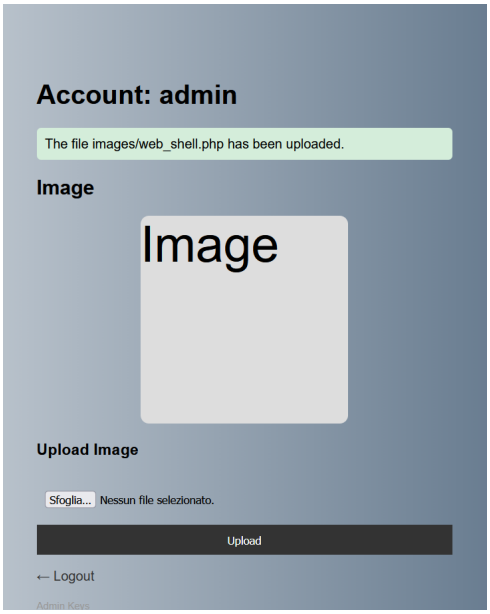


Figure 3: Caricamento del file web_shell.php tramite interfaccia web

Una volta caricato il file, è possibile accedervi direttamente tramite browser. Il server Apache esegue il file come script PHP, permettendo l'esecuzione di comandi arbitrari sul sistema.

MASTER_KEY:5B6Y84SB1SEYEKF2680463SGB

Figure 4: Esecuzione della web shell in una nuova scheda del browser

Approccio 2: Sfruttamento del Path Traversal con Burp Suite

Utilizzando Burp Suite, è possibile intercettare la richiesta di upload e modificare il nome del file includendo sequenze URL-encoded come `..%2f` per bypassare il filtro di sanitizzazione.

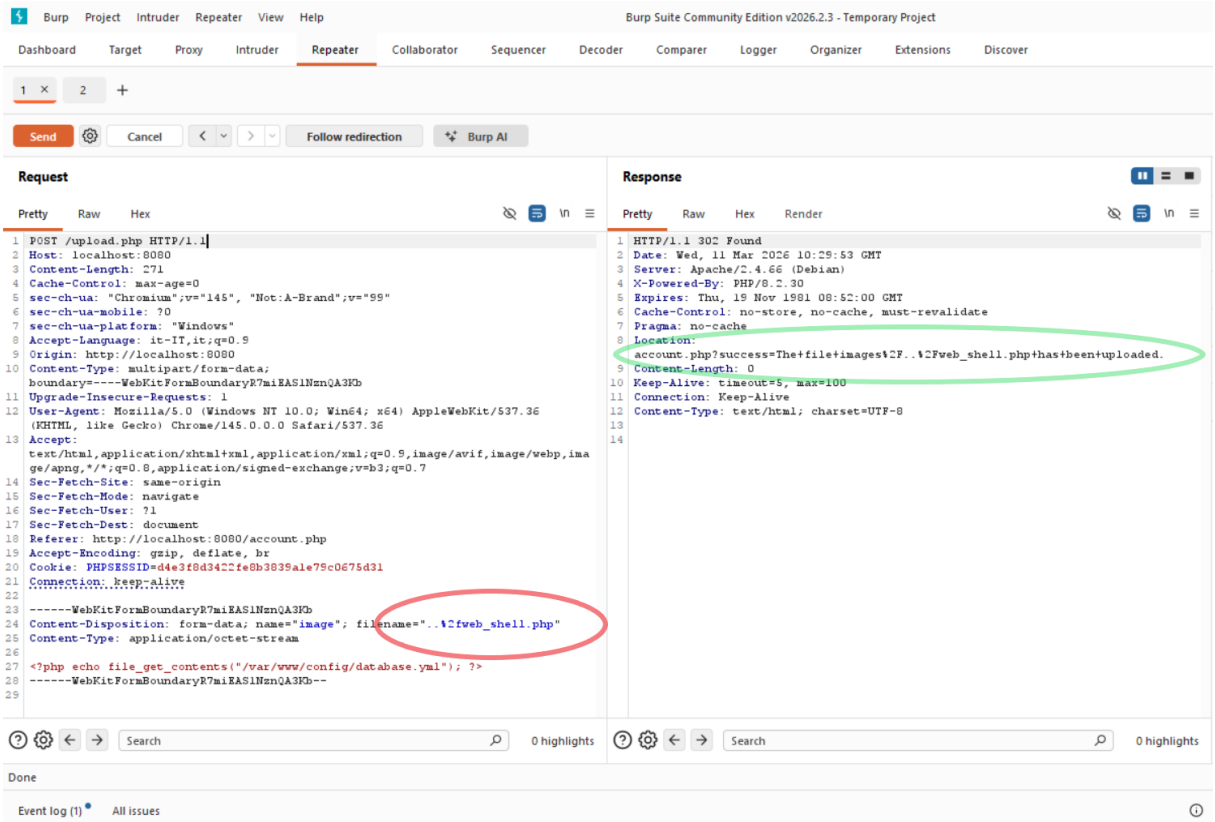


Figure 5: Modifica della richiesta POST tramite Burp Suite per eseguire path traversal

La richiesta modificata permette di posizionare la web shell in una directory al di fuori di `files/images/`, ad esempio nella root dell'applicazione web. Successivamente, tramite una richiesta GET, è possibile eseguire la web shell e leggere il contenuto del file `/var/www/config/database.yml`.

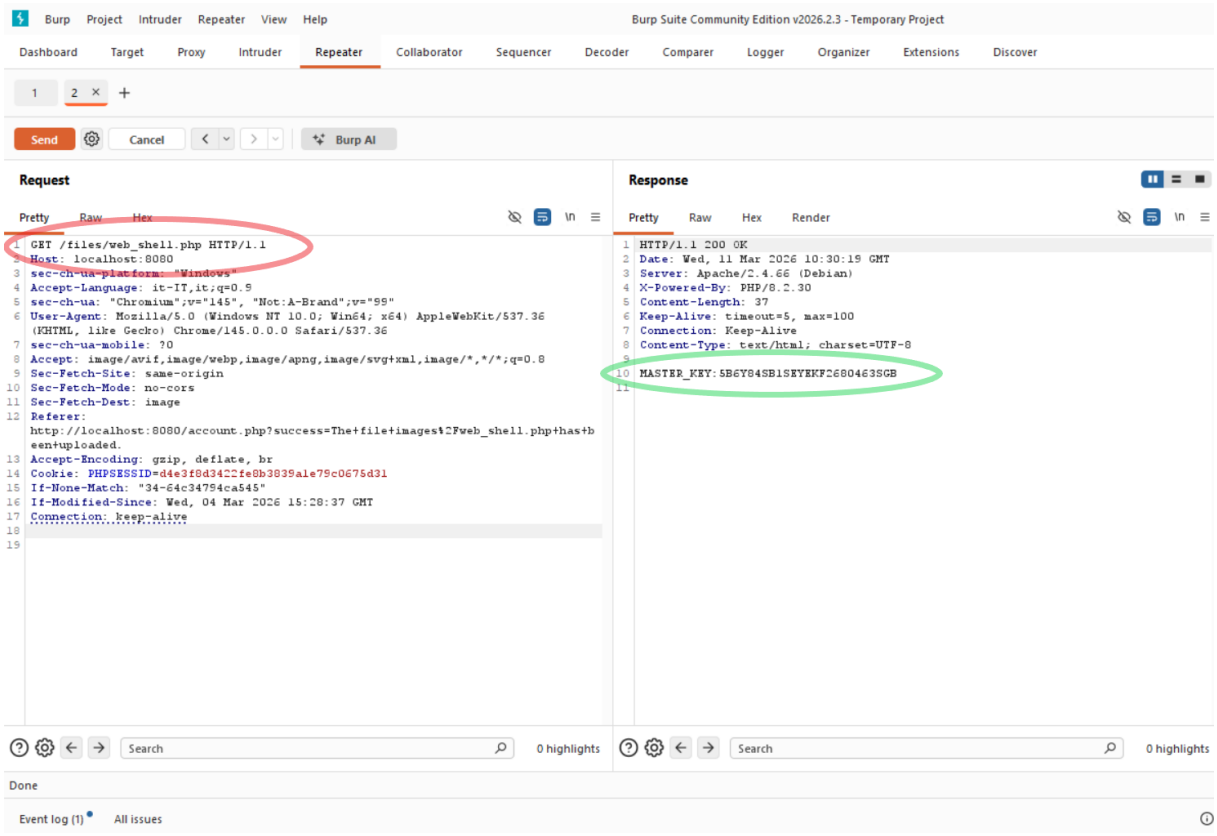


Figure 6: Richiesta GET per l'esecuzione della web shell e recupero della Master Key

Caso medio

A differenza del caso base, in questo scenario l'applicazione implementa numerose contromisure per prevenire gli attacchi di file upload. In particolare, sono stati introdotti i seguenti controlli di sicurezza:

- **Whitelist delle estensioni**: validazione che accetta esclusivamente file con estensioni `.jpg`, `.jpeg`, `.png`, `.gif`
- **Validazione del MIME type**: verifica del tipo di contenuto tramite `finfo_file()` [2]
- **Utilizzo di basename()**: prevenzione degli attacchi di path traversal mediante normalizzazione del percorso
- **Limite dimensionale**: restrizione sulla dimensione massima dei file caricabili

Questi controlli sono comunemente raccomandati nella documentazione sulla sicurezza web e sono generalmente considerati sufficienti per prevenire le forme più comuni di attacchi di file upload.

Tuttavia, l'applicazione rimane vulnerabile a una tecnica di attacco più sofisticata basata sui cosiddetti *file poliglotti*. Un file poliglotta è un file che può essere interpretato come valido da più formati. Nel contesto della sicurezza web, è possibile creare un file che venga riconosciuto come immagine valida, ma che contenga al suo interno codice PHP eseguibile.

La vulnerabilità risiede in una configurazione errata del file `.htaccess`. Mentre il file prevede il blocco dell'esecuzione per i file con estensione `.php`, esso permette esplicitamente l'esecuzione di file con doppia estensione del tipo `.php.jpg`:

```
# Blocca l'esecuzione di file .php standard
<FilesMatch "\.php$">
    Require all denied
</FilesMatch>

# Vulnerabilità: permette l'esecuzione di file con doppia estensione
<FilesMatch "\.php\.(jpg|jpeg|png|gif)$">
    Require all granted
    SetHandler application/x-httpd-php
</FilesMatch>
```

Combinando questa misconfiguration con la possibilità di creare file poliglotti, un attaccante può bypassare tutti i controlli di sicurezza implementati.

Il codice di validazione presenta inoltre una debolezza nell'estrazione dell'estensione del file:

```
<?php
// Estrazione dell'estensione tramite pathinfo
$extension = pathinfo($filename, PATHINFO_EXTENSION);
```

[2] Questa funzione viene utilizzata per ottenere informazioni su un file. Restituisce una descrizione testuale del contenuto dell'argomento passato.

```php
$extension = strtolower($extension);

// Verifica se l'estensione è nella whitelist
if(!in_array($extension, $allowed_extensions)) {
    // Blocca il file
}
?>
```

La funzione `pathinfo()` con il parametro `PATHINFO_EXTENSION` restituisce solo l'ultima estensione del nome file. Pertanto, per un file chiamato `shell.php.jpg`, la funzione restituisce `jpg`, che supera il controllo della whitelist. Tuttavia, il server web Apache, a causa della configurazione dell'`.htaccess`, eseguirà il file come PHP.

Creazione di file poliglotti

Per sfruttare questa vulnerabilità è necessario creare un file che soddisfi contemporaneamente i requisiti di un'immagine valida e contenga codice PHP eseguibile. Sono state utilizzate due tecniche:

1. Inserimento manuale dei magic bytes

Questa tecnica prevede la creazione di un file che inizi con i magic bytes di un formato immagine (ad esempio, `FF D8 FF` per JPEG), seguiti dal codice PHP malevolo. Il controllo del MIME type viene ingannato dai magic bytes iniziali, mentre il parser PHP ignora i byte non validi e esegue il codice trovato.

2. Modifica metadati EXIF con ExifTool

Una tecnica più automatizzata che sfrutta il tool `exiftool` per inserire codice PHP all'interno dei metadati EXIF di un'immagine legittima:

```bash
# Inserimento di una web shell nei metadati EXIF
exiftool -Comment='<?php system($_GET["cmd"]); ?>' foto.jpg -o shell.php.jpg
```

Il file risultante è un'immagine JPEG valida, che supera tutti i controlli di validazione, ma contiene codice PHP eseguibile nei metadati. Quando Apache elabora il file come PHP (a causa della doppia estensione e della configurazione dell'`.htaccess`), il codice viene estratto ed eseguito.

Sfruttamento delle vulnerabilità

Lo sfruttamento della vulnerabilità può essere effettuato mediante due approcci distinti, entrambi basati sul caricamento di file poliglotti con doppia estensione.

Approccio 1: Caricamento diretto di file poliglotta

In questo scenario, viene creato manualmente un file poliglotta con estensione `.php.jpg` e caricato attraverso l'interfaccia web dell'applicazione. Una volta che il file viene accettato e salvato sul server, è possibile accedervi direttamente via browser. Il server Apache, riconoscendo l'estensione `.php` nel nome del file, lo esegue come script PHP.

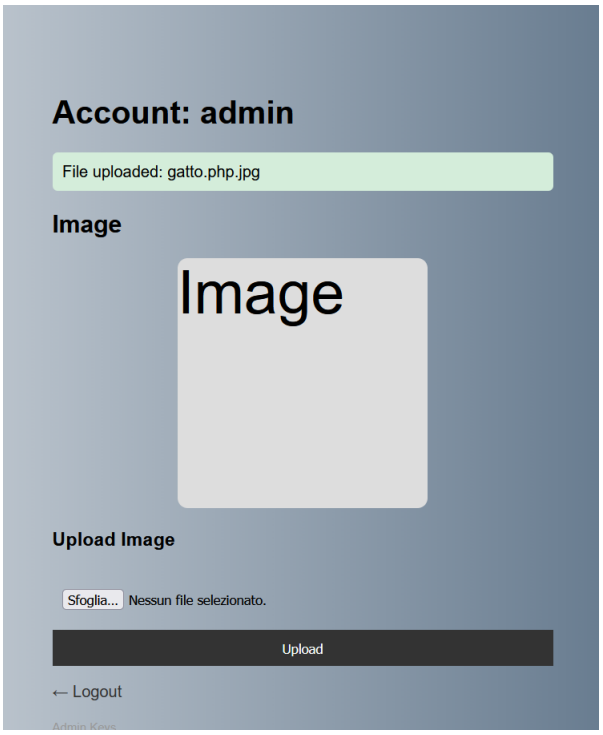


Figure 7: Caricamento del file poliglotta tramite interfaccia web

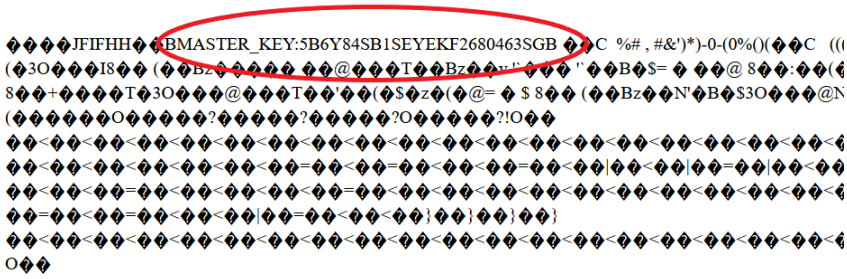


Figure 8: Apertura immagine in altra finestra e recupero della Master Key

Approccio 2: Iniezione tramite metadati EXIF e Burp Suite

Il secondo approccio prevede l'utilizzo di `exiftool` per creare un'immagine con codice PHP incorporato nei metadati EXIF. Per rendere più efficace l'attacco e bypassare eventuali filtri aggiuntivi, viene utilizzato Burp Suite per intercettare e modificare le richieste HTTP.

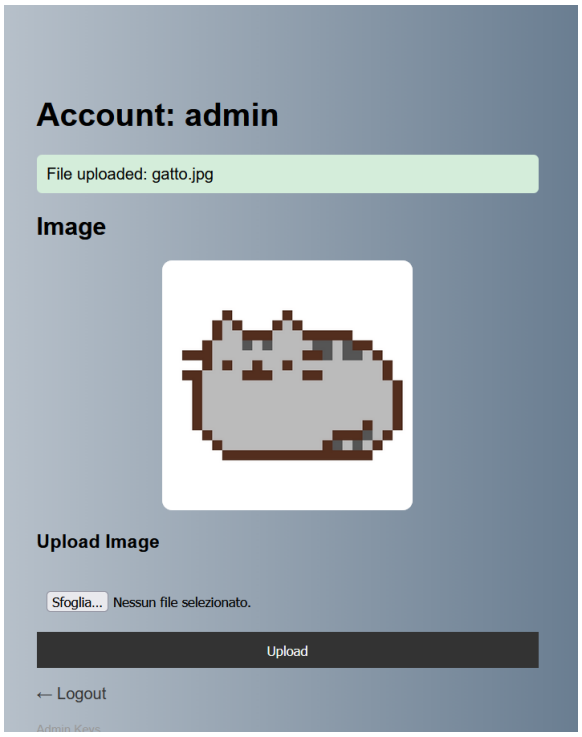


Figure 9: Caricamento dell'immagine con metadati modificati

Attraverso Burp Suite è possibile testare diversi payload e parametri per identificare la configurazione ottimale che permette l'esecuzione del codice PHP incorporato.

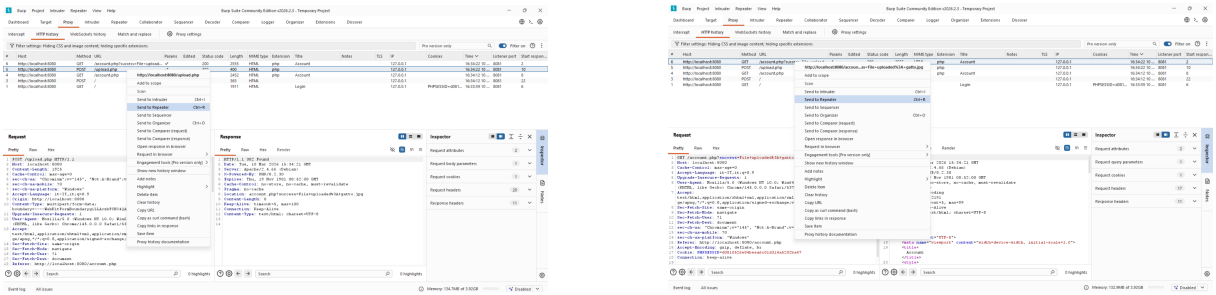


Figure 10: Utilizzo del Repeater di Burp Suite per modificare le richieste POST e GET

Figure 11: Richiesta POST per il caricamento del file

Figure 12: Richiesta GET per l'esecuzione della web shell e recupero della Master Key

In entrambi gli approcci, una volta ottenuta l'esecuzione del codice PHP, l'attaccante può leggere il contenuto del file `/var/www/config/database.yml` e recuperare la Master Key, completando l'obiettivo della challenge.